Oracular Spectacular 풀이 교과서

1장. 용어 사전

이 문서에서 사용하는 모든 용어를 먼저 정리합니다.

1.1 기본 용어

암호화 (Encryption)

평범한 글(평문)을 아무나 못 읽게 뒤섞는 것.

- 비유: 편지를 금고에 넣고 잠그는 것
- "안녕하세요" → "a3f2b1c9..." (알 수 없는 글자로 변함)

복호화 (Decryption)

암호화된 글을 다시 원래대로 되돌리는 것.

- 비유: 금고를 열쇠로 열어서 편지를 꺼내는 것
- "a3f2b1c9..." → "안녕하세요"

키 (Key)

암호화/복호화에 쓰는 비밀번호.

- 비유: 금고의 열쇠
- 키를 모르면 암호를 풀 수 없음

평문 (Plaintext)

암호화하기 전의 원본 메시지.

- 예: "hello world"

암호문 (Ciphertext)

암호화된 후의 메시지. 사람이 읽을 수 없음.

- 예: "a3f2b1c9e8d7..."

바이트 (Byte)

컴퓨터가 정보를 저장하는 단위. 0~255 사이의 숫자 하나.

- 영어 알파벳 하나 = 1바이트
- 'A' = 65, 'a' = 97, '0' = 48

hex (16진수)

숫자를 0~9와 a~f로 표현하는 방법. 바이트 하나를 2글자로 표현.

- 예: 255 = "ff", 0 = "00", 72 = "48"
- "0123456789abcdef" ← 이 16개 글자만 사용

XOR (배타적 논리합)

두 숫자를 비트 단위로 비교하는 연산. 같으면 0, 다르면 1.

- 비유: 두 스위치가 같은 방향이면 꺼짐(0), 다른 방향이면 켜짐(1)
- 특징: A XOR B XOR B = A (두 번 하면 원래로 돌아옴)
- 암호학에서 매우 많이 쓰임

1.2 AES 관련 용어

AES (Advanced Encryption Standard)

전 세계에서 가장 많이 쓰는 암호화 방법.

- 16바이트(128비트)짜리 블록 단위로 암호화
- 키를 알면 암호화/복호화 가능, 모르면 불가능
- 비유: 매우 튼튼한 금고. 열쇠(키) 없이는 절대 못 열음

블록 (Block)

AES가 한 번에 처리하는 데이터 덩어리. 항상 16바이트.

- "hello world!!!!!" ← 16글자 = 1블록
- 메시지가 길면 여러 블록으로 나눠서 처리

CBC 모드 (Cipher Block Chaining)

블록들을 체인처럼 엮어서 암호화하는 방식.

```
[평문1] --XOR--> [AES암호화] --> [암호문1]
   ^                                |
   |                                v
  [IV]                  [평문2] --XOR--> [AES암호화] --> [암호문2]
                           ^
                           |
                        [암호문1]  (앞 블록의 암호문이 다음 블록에 영향)
```

- 같은 평문이라도 IV가 다르면 다른 암호문이 나옴
- 블록끼리 연결되어 있어서 하나 바꾸면 뒤에도 영향

IV (Initialization Vector, 초기화 벡터)

CBC 모드에서 첫 번째 블록에 XOR하는 랜덤 값.

- 매번 다른 IV를 사용 → 같은 메시지도 매번 다른 암호문
- 비유: 요리할 때 처음에 넣는 양념. 양념이 다르면 같은 재료도 다른 맛

중간값 (Intermediate Value)

AES로 복호화한 직후, 아직 XOR하기 전의 값.

- 복호화 과정: 암호문 → [AES복호화] → **중간값** → [XOR with 이전블록] → 평문
- 중간값 = 평문 XOR 이전블록
- 이 값을 알면 평문을 알 수 있음 (이전 블록은 이미 알고 있으니까)

1.3 패딩 관련 용어

패딩 (Padding)

블록 크기(16바이트)에 맞추기 위해 빈 공간을 채우는 것.

- 비유: 택배 상자에 물건 넣고 남는 공간에 스티로폼 채우는 것
- 12바이트 메시지 → 4바이트 패딩 추가 → 16바이트

PKCS7 패딩

패딩의 규칙. 부족한 바이트 수를 그 숫자로 채움.

```
부족한 게 1바이트 → 01
부족한 게 2바이트 → 02 02
부족한 게 3바이트 → 03 03 03
부족한 게 4바이트 → 04 04 04 04
...
```

- 검증: 마지막 바이트를 읽고, 그 숫자만큼 뒤에서부터 확인
- 틀리면 "패딩 오류!" 발생

패딩 오라클 (Padding Oracle)

"이 데이터의 패딩이 맞는지 틀린지" 알려주는 존재.

- 비유: 자물쇠에 열쇠를 넣으면 "맞아요/틀려요" 대답해주는 도우미
- 이 정보만으로도 암호를 풀 수 있음! (패딩 오라클 공격)

패딩 오라클 공격

패딩이 맞는지/틀린지 반복해서 물어보면서 암호를 풀어내는 기법.

1. 암호문의 마지막 바이트를 0~255까지 바꿔가며 질문
2. "패딩 맞아요!"라고 대답하는 값을 찾으면 → 그 바이트의 중간값을 알 수 있음
3. 중간값을 알면 → 평문을 역산 가능

4. 뒤에서부터 한 바이트씩 풀어나감

1.4 이 문제 특수 용어

노이즈 오라클 (Noisy Oracle)

거짓말을 섞어서 대답하는 오라클.

- 일반 오라클: 항상 정확한 답 ("맞아요" / "틀려요")
- **이 문제의 오라클**: 60% 확률로 거짓말!
  - 패딩 맞을 때: "맞아요" 40%, "틀려요" 60%
  - 패딩 틀릴 때: "맞아요" 60%, "틀려요" 40%
- 비유: 10번 물어보면 6번은 거짓말하는 도우미

LLR (Log-Likelihood Ratio, 로그 우도비)

"이 후보가 정답일 가능성"을 숫자로 나타낸 점수.

- 양수: 정답일 가능성 높음
- 음수: 정답 아닐 가능성 높음
- 0: 모르겠음
- 오라클에게 물어볼 때마다 점수가 업데이트됨:
  - 오라클이 "맞아요" → LLR에 -0.405 (정답에게는 불리한 응답)
  - 오라클이 "틀려요" → LLR에 +0.405 (정답에게는 유리한 응답)

쿼리 (Query)

오라클에게 한 번 질문하는 것.

- 이 문제에서는 최대 12,000번 질문 가능
- 32바이트를 풀어야 하므로 → 바이트당 ~370번 질문 가능

캐스케이딩 에러 (Cascading Error, 연쇄 오류)

한 바이트가 틀리면 그 앞의 모든 바이트가 연쇄적으로 틀리는 현상.

- 바이트 14를 공격할 때 바이트 15의 중간값이 필요
- 바이트 15가 틀리면 → 바이트 14도 틀림 → 13도 → ... → 0까지 전부 틀림
- 비유: 도미노. 하나 쓰러지면 뒤에 전부 쓰러짐

2장. 문제 설명

2.1 상황

CryptoHack이라는 암호학 CTF 사이트에 "Oracular Spectacular"라는 문제가 있습니다.

서버에 접속하면:

1. 서버가 랜덤 메시지를 만듦 (16바이트 랜덤 → hex로 변환 → 32글자)
2. 서버가 비밀 키로 AES-CBC 암호화
3. 우리에게 3가지 기능 제공:
    - **encrypt**: 암호문 받기 (무료, 무제한)
    - **unpad**: "이 데이터의 패딩 맞아?" 질문하기 (최대 12,000번)
    - **check**: "메시지가 이거야?" 정답 제출

2.2 목표

서버의 비밀 메시지 32글자를 맞추면 FLAG(정답 코드)를 받음.

2.3 왜 어려운가: 거짓말하는 오라클

일반적인 패딩 오라클 공격은 쉽습니다. 오라클이 항상 정직하니까요.

하지만 이 문제의 오라클은 **60% 확률로 거짓말**합니다!

```
일반 오라클:          이 문제의 오라클:
질문 → 정확한 답      질문 → 60%로 거짓 답
                             40%로 진짜 답
```

비유로 설명하면:

> 보물상자 10개 중 1개에 보물이 있습니다. 도우미에게 "이 상자에 보물 있어?"라고 물어볼 수 있는데, 도우미가 10번 중 6번은 거짓말을 합니다. 총 12,000번 물어볼 수 있고, 정답을 맞춰야 합니다.

3장. 풀이의 핵심 아이디어

3.1 여러 번 물어보면 진실이 보인다

거짓말쟁이라도 같은 질문을 100번 하면?

- 보물이 **있는** 상자: "있어" 40번, "없어" 60번
- 보물이 **없는** 상자: "있어" 60번, "없어" 40번

"있어"라고 대답한 비율(True Rate)을 보면:

- 진짜 보물 상자: **40%** (낮음)
- 가짜 상자: **60%** (높음)

"있어" 비율이 가장 낮은 상자가 진짜 보물 상자!

3.2 문제: 질문 횟수가 부족함

- 32바이트 × 16후보 = 512개를 구분해야 함
- 12,000번 질문 가능

- 바이트당 370번, 후보당 **23번**밖에 못 물어봄
- 23번으로 40% vs 60% 구분? → 어렵지만 불가능하지는 않음

3.3 질문을 똑똑하게 분배하기

16개 후보에게 23번씩 균등하게 물어보면 정확도가 낮습니다.

더 좋은 방법: **유망한 후보에게 집중 질문!**

```
라운드 1: 16명에게 4번씩 물어봄 (64번 사용)
        → 가장 유망한 2명 선택

라운드 2: 그 2명에게 나머지 306번 집중!
        → 충분한 데이터로 정확한 판단
```

이것이 **Adaptive Top-2** 전략입니다.

4장. 이전 시도들과 실패 원인

4.1 시도 1~2: 한 명만 계속 질문하기

방법: 가장 유망한 후보 1명에게만 계속 질문

문제: 처음에 운 좋게 1위가 된 가짜 후보에게 질문이 몰림. 가짜라는 걸 깨달을 때쯤 이미 질문 횟수를 다 씀.

```
가짜 후보: 운 좋게 1위 등극 → 수백 번 질문 → 가짜 판명 → 이미 늦음!
진짜 후보: 질문 한 번도 못 받음 → 영원히 묻힘
```

비유: 면접에서 첫인상 좋은 사람만 계속 면접하다가 시간 다 씀

4.2 시도 3~4: 단계별 탈락 (Sequential Halving)

방법:

- 라운드 1: 16명 → 8명 탈락
- 라운드 2: 8명 → 4명 탈락
- 라운드 3: 4명 → 2명 탈락
- 최종: 2명 중 승자 선택

문제: 진짜 정답이 초반 라운드에서 탈락하면 **복구 불가능**!

```
라운드 1에서 7번만 질문하면:
진짜 후보도 운 나쁘면 "맞아" 대답을 많이 받을 수 있음 (60%가 정상인데 운 나쁘면 70%)
```

```
  → 가짜로 오해받아 탈락!
  → 한번 탈락하면 되돌릴 수 없음
```

측정 결과: **바이트당 정확도 75%**

비유: 오디션에서 1차 예선이 너무 짧아서 실력자가 탈락하는 것

4.3 시도 5~6: 탈락 조건 조정

탈락 기준을 느슨하게/엄격하게 바꿔봤지만 근본적인 문제는 동일:

- 느슨하면 → 가짜들이 너무 많이 살아남아 질문 낭비
- 엄격하면 → 진짜가 탈락할 위험 증가

4.4 시도 7~8: 이론적 분석 + 독립 추적

수학적으로 최적 배분을 계산하고, 후보 간 독립적으로 점수 추적.

문제: 이론과 실제의 괴리

- 이론: "바이트당 99.3% 정확도" → 계산상 성공률 80%
- 실제: 바이트당 ~85% → 성공률 ~0.5%
- 원인: **selection bias** (탈락에서 살아남은 가짜는 평균보다 강함)를 이론이 무시

4.5 모든 시도의 공통 실패 원인

원인 1: "탈락"은 되돌릴 수 없다

Sequential Halving에서 한번 탈락시키면 끝. 실수를 교정할 방법이 없음.

원인 2: 정확도를 과대평가

이론상 99%라고 생각했지만 실제로는 75~85%. 이 차이가 32바이트에 걸쳐 기하급수적으로 커짐.

원인 3: 한 번에 성공하려 함

0.85^32 = 0.7%. **한 번에 32바이트 전부 맞출 확률은 1% 미만.** 이건 알고리즘의 한계가 아니라 수학적 현실. 재시도가 필수.

5장. 최종 풀이: Adaptive Top-2

5.1 핵심 아이디어: "탈락" 대신 "자연 도태"

Sequential Halving의 문제는 "영구 탈락"이었습니다.

Adaptive Top-2는 다릅니다: **아무도 탈락시키지 않습니다.**

대신:

1. 가장 유망한 2명에게 질문을 집중
2. 가짜가 1위라면? → 질문할수록 점수가 떨어짐 (자연 도태)
3. 진짜가 결국 떠오름

```
[가짜A] 점수 높음 → 질문 받음 → 점수 하락 → 질문 받음 → 점수 하락
[가짜B] 점수 높음 → 질문 받음 → 점수 하락
[진짜C] 점수 낮음 → 질문 안 받음 → 점수 유지... → 1위 등극! → 질문 받음 → 점수 상승!
```

5.2 왜 가짜는 떨어지고 진짜는 올라갈까?

LLR 점수는 질문할 때마다 변합니다:

진짜 후보에게 질문하면:

- 40% 확률로 -0.405 (잠깐 떨어짐)
- 60% 확률로 +0.405 (올라감)
- 평균: **+0.081** (서서히 올라감)

가짜 후보에게 질문하면:

- 60% 확률로 -0.405 (떨어짐)
- 40% 확률로 +0.405 (잠깐 올라감)
- 평균: **-0.081** (서서히 떨어짐)

질문을 많이 할수록 진짜는 올라가고, 가짜는 내려갑니다!

5.3 전체 과정 (한 바이트)

```
=== 스크리닝 (64쿼리) ===
16개 후보에게 각각 4번씩 질문
→ 초기 LLR 점수 계산

=== 적응적 집중 (~306쿼리) ===
반복:
  1. 전체 16명 중 점수 상위 2명 선택
  2. 그 2명에게 각각 1번 질문
  3. 점수 업데이트
  4. 다시 1번으로

=== 최종 선택 ===
가장 높은 LLR 점수의 후보 = 정답
```

5.4 구체적 예시

```
후보:  0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
정답: '7' (우리는 모름)
```

```
--- 스크리닝 (4번씩 질문) ---
점수: -0.8 0.0 -0.8 0.0 0.8 0.0 -0.8 [1.6] 0.0 -0.8 0.0 0.0 0.8 0.0 -0.8 0.0
                                         ↑ 진짜 '7'이 운 좋게 1위

→ 상위 2: '7'(1.6)과 '4'(0.8)

--- 적응적 집중 ---
'7'(진짜)에게 질문 → 점수 서서히 상승: 1.6 → 2.4 → 3.2 → ...
'4'(가짜)에게 질문 → 점수 서서히 하락: 0.8 → 0.0 → -0.8 → ...

100번 후: '7' = +9.7, '4' = -7.3
→ '7' 확정 승리!
```

만약 스크리닝에서 진짜가 1위가 아니었다면?

```
--- 스크리닝 ---
점수: '4'=1.6, '2'=0.8, '7'(진짜)=-0.8, ...

→ 상위 2: '4'와 '2' (둘 다 가짜!)

--- 적응적 집중 ---
'4'에게 질문 → 하락: 1.6 → 0.8 → 0.0 → -0.8
'2'에게 질문 → 하락: 0.8 → 0.0 → -0.8

20번 후: '4'=-1.6, '2'=-1.6
'7'(진짜)은 질문 안 받았으므로 점수 그대로 -0.8

→ '7'이 이제 1위! (가짜들이 자연 도태됨)
→ 이제 '7'에게 집중 질문 → 점수 상승 → 최종 승리
```

이것이 Adaptive Top-2의 핵심: 가짜는 질문받을수록 떨어지고, 진짜는 자연스럽게 부상합니다.

5.5 Sequential Halving과의 비교

	Sequential Halving	**Adaptive Top-2**
방식	탈락시킴 (되돌릴 수 없음)	자연 도태 (되돌릴 수 있음)
진짜가 초반에 운 나쁘면	영구 탈락 → 실패	가짜가 떨어질 때까지 대기 → 부상
가짜가 초반에 운 좋으면	끝까지 살아남을 수 있음	질문받을수록 떨어짐
바이트당 정확도	75%	**86%**

6장. 캐스케이딩과 재시도

6.1 캐스케이딩 에러란?

패딩 오라클 공격은 뒤에서부터 한 바이트씩 풀어갑니다:

```
바이트:  15 → 14 → 13 → 12 → ... → 1 → 0
         먼저   →   →   →   →   →   →  마지막
```

바이트 14를 풀 때, 바이트 15의 정답이 필요합니다. 바이트 13을 풀 때, 바이트 14와 15의 정답이 필요합니다.

만약 바이트 14가 틀리면?

```
바이트 15: 맞음 ✓
바이트 14: 틀림 ✗ ← 여기서 오류!
바이트 13: 14가 틀려서 풀 수 없음 ✗
바이트 12: 마찬가지 ✗
...
바이트 0:  마찬가지 ✗
```

도미노처럼 하나 틀리면 그 앞 전부 틀립니다!

6.2 이것이 "재시도"가 필요한 이유

바이트당 86% 정확도면 아주 높아 보이지만:

```
1바이트 맞을 확률:  86%
2바이트 연속 맞을 확률:  86% × 86% = 74%
8바이트 연속:  86%^8 = 30%
16바이트 연속 (1블록):  86%^16 = 8.4%
32바이트 연속 (전체 메시지):  86%^32 = 0.7%
```

한 번에 성공할 확률: 0.7%

하지만 여러 번 시도하면:

```
100번 시도:  1 - (1-0.007)^100 = 50% 성공
300번 시도:  1 - (1-0.007)^300 = 88% 성공
500번 시도:  1 - (1-0.007)^500 = 97% 성공
```

500번 시도하면 97% 확률로 성공!

6.3 재시도가 가능한 이유

매번 새로운 접속을 하면:

- 새로운 랜덤 메시지
- 새로운 랜덤 키
- 새로운 12,000번 질문 기회

이전 실패가 다음 시도에 전혀 영향 없음. 독립적인 시도.

비유: 복권을 여러 장 사는 것과 같음. 한 장의 당첨 확률은 낮지만, 수백 장 사면 당첨 가능.

7장. 실험 결과

7.1 전략별 바이트 정확도 비교

16개 후보 중 정답을 맞출 확률 (370쿼리, 1000회 측정):

```
전략                        정확도
──────────────────────────────────
균등 질문 (탈락 없음)        59.0%
Sequential Halving          75.1%
SH Heavy R0                 67.0%
SH 5라운드                  73.0%
적응적 제거                 58.6%
★ Adaptive Top-2            85.7%  ← 최고!
```

Adaptive Top-2가 다른 모든 전략을 압도합니다.

7.2 로컬 테스트 결과

컴퓨터에서 서버를 시뮬레이션하여 테스트:

```
317회 시도 중 3회 성공 (약 0.95%)
평균 ~100-200회 시도에 1회 성공

성공 기록:
  #1: 216번째 시도 (88초)
  #2: 312번째 시도 (126초)
  #3: 317번째 시도 (128초)

복구된 메시지: dab444d96828292b02fd7aa61273667c
실제 메시지:   dab444d96828292b02fd7aa61273667c  ← 완벽 일치!
```

7.3 서버 실행

실제 CryptoHack 서버에서:

- 1회 시도 = 12,000 쿼리 × 네트워크 왕복 = 수 분
- 5개 병렬 실행으로 속도 향상
- 예상 소요 시간: 수 시간

8장. 전체 요약

한 줄 요약

> 거짓말쟁이 도우미에게 12,000번 질문해서 32글자 비밀 메시지를 맞추는 문제. 가장 유망한 2명에게 집중 질문하는 전략으로 바이트당 86% 정확도를 달성하고, 수백 번 재시도하여 풀었다.

핵심 교훈

1. **거짓말쟁이도 여러 번 물어보면 진실이 드러난다** (통계의 힘)
2. **"탈락"보다 "자연 도태"가 안전하다** (Adaptive Top-2 > Sequential Halving)
3. **한 번에 성공하려 하지 말고, 재시도를 설계에 포함시켜라**
4. **이론적 분석을 맹신하지 말고, 실험으로 검증하라** (이론 99% vs 실제 75~86%)

9장. 코드 해설

9.1 문제 서버 코드 (13423_*.py)

서버에서 돌아가는 코드입니다. 우리가 공격해야 할 대상.

초기 설정

```
FLAG = 'crypto{??????????????????????????????????????????????????}'
rng = SystemRandom()
```

- `FLAG`: 우리가 얻고 싶은 정답 코드
- `rng`: 암호학적으로 안전한 난수 생성기 (거짓말 확률 결정에 사용)

Challenge 클래스 초기화

```
class Challenge:
    def __init__(self):
        self.before_input = "That last challenge was pretty easy..."
        self.message = urandom(16).hex()    # ← 우리가 맞춰야 할 메시지!
        self.key = urandom(16)               # ← 비밀 키 (우리는 모름)
        self.query_count = 0
        self.max_queries = 12_000            # ← 최대 12,000번 질문 가능
```

- `urandom(16)`: 16바이트 랜덤 데이터 생성
- `.hex()`: hex 문자열로 변환 → 32글자 (예: "a3f2b1c9e8d7...")
- 메시지의 각 글자는 0~9, a~f 중 하나 → **후보가 16개**

encrypt 기능: 암호문 받기

```
def get_ct(self):
    iv = urandom(16)                                    # 랜덤 IV 생성
    cipher = AES.new(self.key, AES.MODE_CBC, iv=iv)     # AES-CBC 암호화 객체
    ct = cipher.encrypt(self.message.encode("ascii"))    # 메시지 암호화
    return {"ct": (iv+ct).hex()}                         # IV + 암호문 반환
```

- 매번 다른 IV → 같은 메시지도 매번 다른 암호문
- 반환값: IV(16바이트) + 암호문(32바이트) = 48바이트 = 96자 hex
- **이 기능은 무료! 몇 번이든 호출 가능**

unpad 기능: 노이즈 패딩 오라클 (핵심!)

```
def check_padding(self, ct):
    ct = bytes.fromhex(ct)
    iv, ct = ct[:16], ct[16:]                           # 앞 16바이트 = IV, 나머지
= 암호문
    cipher = AES.new(self.key, AES.MODE_CBC, iv=iv)
    pt = cipher.decrypt(ct)                              # 복호화 (패딩 제거 안 함)
    try:
        unpad(pt, 16)                                    # PKCS7 패딩 검증
    except ValueError:
        good = False                                     # 패딩 틀림
    else:
        good = True                                      # 패딩 맞음
    self.update_query_count()
    return {"result": good ^ (rng.random() > 0.4)}      # ← 60% 확률로 뒤집기!
```

한 줄씩 해석:

1. 우리가 보낸 hex 문자열을 바이트로 변환
2. 앞 16바이트를 IV로, 나머지를 암호문으로 분리
3. AES-CBC로 복호화 → 평문 얻음
4. PKCS7 패딩이 올바른지 검증
5. **핵심**: `good ^ (rng.random() > 0.4)`
    - `rng.random() > 0.4` → 60% 확률로 True
    - `good ^ True` = `not good` (뒤집힘!)
    - `good ^ False` = `good` (그대로)
    - 결과: **60% 확률로 답이 뒤집힌다**

check 기능: 정답 제출

```
def check_message(self, message):
    if message != self.message:
        self.exit = True                    # 틀리면 연결 종료!
```

```
        return {"error": "incorrect message"}
    return {"flag": FLAG}                    # 맞으면 FLAG 획득!
```

- 딱 한 번의 기회. 틀리면 바로 연결 끊김
- 맞으면 FLAG 반환 → 문제 해결!

9.2 솔버 코드 (solver.py)

상수 정의

```
HEX_BYTES = [ord(c) for c in '0123456789abcdef']
# = [48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 97, 98, 99, 100, 101, 102]
# 메시지의 각 글자가 될 수 있는 16개 후보의 ASCII 코드값

LOG_P = math.log(0.4 / 0.6)  # = -0.405
# LLR 점수 업데이트에 사용하는 상수
# 오라클이 True 응답 → 이 값을 더함 (음수 = 정답에게 불리)
# 오라클이 False 응답 → 이 값을 뺌 (양수 = 정답에게 유리)
```

로컬 오라클: 서버를 컴퓨터에서 시뮬레이션

```
class LocalOracle:
    def __init__(self):
        self.message = urandom(16).hex()    # 랜덤 32자 메시지 생성
        self.key = urandom(16)               # 랜덤 키 생성
        self.query_count = 0
```

- 서버 코드와 동일한 동작을 로컬에서 재현
- 네트워크 없이 빠르게 테스트 가능 (0.4초 vs 수분)

```
    def check_padding(self, ct_hex):
        # ... (서버와 동일한 복호화 + 패딩 검증) ...
        return good ^ (rng.random() > 0.4)    # 동일한 60% 노이즈
```

서버 오라클: 실제 서버와 통신

```
class ServerOracle:
    def __init__(self, host='socket.cryptohack.org', port=13423):
        self.s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # TCP 소켓 생
성
        self.s.connect((host, port))                                  # 서버에 접속
        self.s.settimeout(180)                                        # 3분 타임아
웃
```

```
        self._recv_line()                                          # 배너 메시지
읽기
```

```
    def _cmd(self, obj):
        self.s.sendall((json.dumps(obj) + "\n").encode())  # JSON으로 명령 전송
        return json.loads(self._recv_line())                # JSON으로 응답 수신
```

- 서버와 JSON 형식으로 대화
- 예: `{"option": "unpad", "ct": "a3f2..."}` → `{"result": true}`

make_ct: 패딩 오라클 공격용 암호문 조립

```
def make_ct(target, inter, pos, pad, cand):
    m = bytearray(os.urandom(16))           # ① 16바이트 랜덤으로 채움
    for k in range(pos + 1, 16):            # ② 이미 푼 바이트들: 패딩값으로 설정
        m[k] = inter[k] ^ pad
    m[pos] = cand ^ pad                     # ③ 지금 테스트할 바이트: 후보값 설정
    return (bytes(m) + bytes(target)).hex()  # ④ 조립한 블록 + 타겟 블록 = 암호문
```

이 함수가 하는 일 (그림으로 설명):

```
우리가 만드는 블록 (m):          타겟 블록 (target):
┌──────────────────────┐        ┌──────────────────────┐
| 랜덤 랜덤 ... |후보|패딩|패딩|   |  원래 암호문 블록 (건드리지 않음)  |
|  (상관없음)    | ③ | ② | ② |   |                                |
└──────────────────────┘        └──────────────────────┘

         ↓                              ↓
     "이전 블록" 역할              AES 복호화 대상

서버가 복호화하면:
  복호화된 바이트[pos] = AES_dec(target)[pos] XOR m[pos]
                       = 중간값[pos] XOR (후보 XOR 패딩값)

  만약 후보 = 진짜 중간값이면:
    복호화 결과 = 중간값 XOR 중간값 XOR 패딩값 = 패딩값 ✓ (패딩 맞음!)

  만약 후보 ≠ 진짜 중간값이면:
    복호화 결과 = 다른값 (패딩 틀림!)
```

- `target`: 공격 대상 암호문 블록 (C1 또는 C2)
- `inter`: 지금까지 알아낸 중간값들
- `pos`: 현재 공격 중인 바이트 위치 (15부터 0까지)
- `pad`: 패딩값 = 16 - pos (마지막 바이트면 1, 그 앞이면 2, ...)
- `cand`: 테스트할 후보 중간값

find_byte: 한 바이트 복구 (Adaptive Top-2 핵심!)

```
def find_byte(oracle_fn, target, inter, pos, prev_byte, budget):
    pad = 16 - pos                              # 패딩값 계산
    cands = [prev_byte ^ h for h in HEX_BYTES]  # 16개 후보 생성
    llr = [0.0] * 16                             # 각 후보의 LLR 점수 (0에서 시작)
    used = 0                                     # 사용한 쿼리 수
```

후보 생성 원리:

```
중간값 = 평문 XOR 이전블록
후보 중간값 = hex문자 XOR 이전블록의 해당 바이트

예: 이전블록[15] = 0x04일 때
  '0'(0x30) → 후보 = 0x30 XOR 0x04 = 0x34
  '1'(0x31) → 후보 = 0x31 XOR 0x04 = 0x35
  ...
  'f'(0x66) → 후보 = 0x66 XOR 0x04 = 0x62
→ 16개 후보 중 하나가 진짜 중간값
```

```
    # === 스크리닝 단계 ===
    for idx in range(16):                           # 16개 후보 각각에 대해
        for _ in range(4):                          # 4번씩 질문
            r = oracle_fn(make_ct(..., cands[idx]))  # 오라클에게 질문
            llr[idx] += LOG_P if r else -LOG_P       # LLR 점수 업데이트
            used += 1
```

LLR 업데이트 원리:

```
오라클 응답이 True일 때:
  llr += log(0.4/0.6) = -0.405
  → "True는 가짜한테 더 잘 나오는 응답이니까, 이 후보가 정답일 가능성 ↓"

오라클 응답이 False일 때:
  llr += -log(0.4/0.6) = +0.405
  → "False는 진짜한테 더 잘 나오는 응답이니까, 이 후보가 정답일 가능성 ↑"

진짜 후보: True 40% + False 60% → 평균 LLR 변화 = +0.081/쿼리 (서서히 상승)
가짜 후보: True 60% + False 40% → 평균 LLR 변화 = -0.081/쿼리 (서서히 하락)
```

```
    # === 적응적 집중 단계 ===
    while used < budget:                                        # 예산 다 쓸 때까지
        sorted_idx = sorted(range(16), key=lambda i: llr[i],
                            reverse=True)                       # LLR 높은 순 정렬
```

```
        for t in sorted_idx[:2]:                                # 상위 2명에게만
            r = oracle_fn(make_ct(..., cands[t]))               # 질문
            llr[t] += LOG_P if r else -LOG_P                    # 점수 업데이트
            used += 1
```

왜 상위 2명에게만 질문하나?

```
LLR 1위: 가장 정답 같은 후보 → 진짜인지 확인 필요
LLR 2위: 1위의 경쟁자 → 얘가 진짜인지도 확인 필요
3위 이하: 이미 점수 낮음 → 질문 낭비

만약 1위가 가짜라면:
  질문할수록 1위의 점수가 하락 → 자연스럽게 순위 하락
  원래 낮았던 진짜가 상대적으로 올라옴 → 새로운 top-2에 진입
```

```
    # === 최종 선택 ===
    best = max(range(16), key=lambda i: llr[i])  # LLR 가장 높은 후보 = 정답
    return cands[best], used
```

solve_attempt: 전체 메시지 복구 (1회 시도)

```
def solve_attempt(oracle, budget_per_byte=370, verbose=False):
    ct_hex = oracle.get_ct()                    # 암호문 받기 (무료)
    ct_b = bytes.fromhex(ct_hex)
    iv = list(ct_b[:16])                        # 앞 16바이트 = IV
    c1 = list(ct_b[16:32])                      # 다음 16바이트 = 블록1 암호문
    c2 = list(ct_b[32:48])                      # 마지막 16바이트 = 블록2 암호문
```

```
암호문 구조 (48바이트 = 96자 hex):
┌──────────┬──────────┬──────────┐
│   IV     │   C1     │   C2     │
│ 16bytes  │ 16bytes  │ 16bytes  │
└──────────┴──────────┴──────────┘
```

```
    # === 블록 2 공격 (메시지 뒷부분 16글자) ===
    inter2 = [0] * 16                           # 중간값 저장소 (0으로 초기화)
    for pos in range(15, -1, -1):               # 바이트 15부터 0까지 역순!
        best, _ = find_byte(
            oracle.check_padding,               # 질문할 함수
            c2,                                 # 공격 대상: 블록2 암호문
            inter2,                             # 지금까지 알아낸 중간값
            pos,                                # 현재 바이트 위치
```

```
            c1[pos],                            # 이전 블록(C1)의 해당 바이트
            budget_per_byte                     # 이 바이트에 쓸 쿼리 수 (370)
        )
        inter2[pos] = best                      # 찾은 중간값 저장
```

왜 15부터 0 순서로?

```
PKCS7 패딩은 마지막 바이트부터 검증:
  pad=1: 바이트15만 확인        → 바이트15부터 공격
  pad=2: 바이트14,15 확인       → 14를 공격할 때 15가 필요
  pad=3: 바이트13,14,15 확인    → 13을 공격할 때 14,15가 필요
  ...
뒤에서부터 풀어야 앞을 풀 수 있음!
```

```
    # === 블록 1 공격 (메시지 앞부분 16글자) ===
    inter1 = [0] * 16
    for pos in range(15, -1, -1):
        best, _ = find_byte(
            oracle.check_padding, c1, inter1, pos, iv[pos],  # 이전 블록 = IV
            budget_per_byte
        )
        inter1[pos] = best
```

```
    # === 평문 복원 ===
    pt1 = bytes([inter1[i] ^ iv[i] for i in range(16)])     # 중간값 XOR IV = 평문1
    pt2 = bytes([inter2[i] ^ c1[i] for i in range(16)])     # 중간값 XOR C1 = 평문2
    return (pt1 + pt2).decode('ascii', 'replace')            # 합쳐서 32글자 메시지
```

평문 복원 원리:

```
중간값 = AES_dec(암호문블록)
평문 = 중간값 XOR 이전블록

블록1: 평문1 = inter1 XOR IV     (CBC 첫 블록은 IV와 XOR)
블록2: 평문2 = inter2 XOR C1     (CBC 두번째 블록은 C1과 XOR)
```

run_server: 자동 재시도 루프

```
def run_server(max_attempts=500):
    for attempt in range(1, max_attempts + 1):     # 최대 N번 시도
        oracle = ServerOracle()                     # 새 접속 (새 메시지, 새 키)
        msg = solve_attempt(oracle, verbose=True)   # 메시지 복구 시도
```

```
        flag = oracle.check_message(msg)            # "이게 정답?" 제출

        if flag:                                    # 맞으면!
            print(f"FLAG: {flag}")                  # FLAG 출력
            with open("flag.txt", "w") as f:
                f.write(flag + "\n")                # 파일에 저장
            return True

        # 틀리면 → 접속 끊고 다음 시도 (새 접속 = 새 메시지)
        oracle.close()
```

재시도가 되는 이유:

```
시도 1: 서버가 메시지 "a3f2..." 생성 → 우리가 풀기 시도 → 실패 → 연결 종료
시도 2: 서버가 메시지 "7b9c..." 생성 → 우리가 풀기 시도 → 실패 → 연결 종료
시도 3: 서버가 메시지 "d1e0..." 생성 → 우리가 풀기 시도 → 성공! → FLAG 획득!

매번 새 메시지, 새 키. 이전 실패와 무관한 독립적인 시도.
시도당 성공률 ~0.7% × 수백 번 = 결국 성공
```

9.3 전체 흐름 한눈에 보기

```
solver.py 실행
    │
    ▼
[시도 1] ── 서버 접속 ── encrypt로 암호문 수신
    │                          │
    │              ┌──────────┘
    │              ▼
    │         IV + C1 + C2 파싱
    │              │
    │    ┌─────────┴─────────┐
    │    ▼                   ▼
    │  블록2 공격            블록1 공격
    │  (C2, 이전블록=C1)     (C1, 이전블록=IV)
    │    │                   │
    │    │  바이트15~0 각각:  │
    │    │  ┌──────────┐     │
    │    │  │ 스크리닝 64q │  │
    │    │  │ Top-2 집중   │  │
    │    │  │ → 중간값 확정│  │
    │    │  └──────────┘     │
    │    │                   │
    │    ▼                   ▼
    │  inter2 완성          inter1 완성
    │    │                   │
    │    └──────────┬────────┘
    │               ▼
```

```
      │       평문 복원 (XOR)
      │            │
      │            ▼
      │       check로 정답 제출
      │            │
      │        ┌───┴───┐
      │        ▼       ▼
      │     성공!     실패...
      │     FLAG!       │
      │                 ▼
      │            [시도 2] → 새 접속부터 반복
      │                 │
      │                 ▼
      │            [시도 3] → ...
      │                 │
      │                 ▼
      │            [시도 N] → 결국 성공!
```

부록: 코드 실행 방법

```
# 로컬 테스트 (알고리즘 동작 확인)
python solver.py local 5000

# 서버 풀이 (자동 재시도)
python solver.py server 5000

# 병렬 실행 (5개 동시)
for i in $(seq 1 5); do
  nohup python3 solver.py server 1000 > result_$i.txt 2>&1 &
done

# 결과 확인
grep "FLAG" result_*.txt
```